Lab 2: Network Delay, Packet Loss, and Throughput with CML and Wireshark

1. Background

In packet-switched networks, the performance of data transmission is governed by three fundamental metrics:

- **Delay (Latency)**: The time it takes for a packet to travel from the source to the destination. The end-to-end delay is the sum of processing delay, queuing delay, transmission delay, and propagation delay.
- **Packet Loss**: The percentage of packets that fail to reach their destination. This typically occurs when a router's buffer overflows due to congestion, causing the router to drop incoming packets.
- **Throughput**: The rate at which data is successfully delivered over a communication channel. Throughput is limited by the bottleneck link along the end-to-end path.

Cisco Modeling Labs (CML) provides **Link Condition** capabilities on links. This allows you to apply simulated impairments (latency, packet loss, and bandwidth limits) directly to the link connecting two nodes to observe and analyze network behavior under different conditions.

AI Policy & Acceptable Use Guidelines

 AI Policy & Acceptable Use Guidelines - Yellow Icon

Artificial Intelligence tools can be powerful learning aids, but they must be used ethically and responsibly to ensure you develop core networking skills.

- **Lab Assignments:** Do not use AI to perform lab steps or write your lab submissions. However, if you get stuck on a concept or technical obstacle, you may use AI as a virtual tutor to ask clarifying questions and help you troubleshoot.

---

2. Objective / Purpose

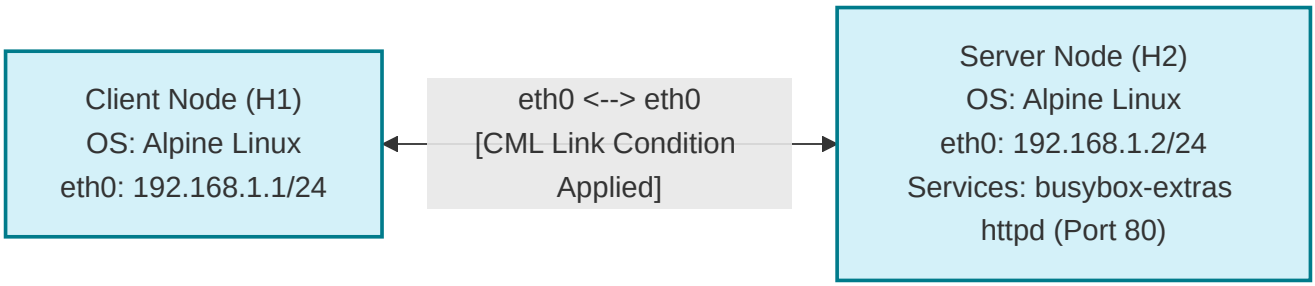
The objective of this lab is to configure, measure, and analyze the direct impacts of network delay, packet loss, and bandwidth constraints using a simulated client-server topology in CML.

Specifically, you will:

- Set up a direct link between two **Alpine Linux** hosts representing a Client and a Server.
- Configure link impairments (latency, packet loss, and bandwidth limits) in CML.
- Use `ping` to measure round-trip time (RTT) and packet loss rates.
- Use BusyBox `httpd` on the server and `wget` on the client to measure throughput under different bandwidth constraints.
- Capture the traffic in Wireshark to observe TCP retransmissions and packet drops.

---

3. CML Topology and Configuration Procedure



Step 1: Deploying the Nodes

1. Log in to the CML UI and create a new lab called `Lab_02_Delay_Loss_Throughput`.
2. Drag two **Alpine Linux** nodes onto the workspace.
3. Rename the first node `<initials>-H1` (this will be the Client) and the second node `<initials>-H2` (this will be the Server).
4. Create a link connecting the two nodes (`eth0` on H1 to `eth0` on H2).

Step 2: Configuring the Client and Server

You will use CML's **Edit Config** tab to inject configuration scripts that execute automatically when the nodes boot.

1. Client Host (H1) Configuration

- Click on the H1 client node in the workspace.
- In the bottom pane, navigate to the **Edit Config** tab.
- Enter the following shell script to configure its static IP address, start the interface, and signal CML that the boot process is complete:

```
#!/bin/sh
echo "=== Starting Alpine H1 Boot Script ===" >/dev/console

# Configure network interface eth0
echo "Configuring eth0 static IP..." >/dev/console
ip addr add 192.168.1.1/24 dev eth0
ip link set eth0 up
ip addr show dev eth0 >/dev/console

# Signal boot completion to CML
echo "READY" >/dev/console
exit 0
```

2. Server Host (H2) Configuration

- Click on the H2 server node in the workspace.
- Navigate to the **Edit Config** tab.
- Enter the following shell script to configure its static IP address, create a large test file, start the built-in lightweight HTTP server, and notify CML that it is ready:

```
#!/bin/sh
echo "=== Starting Alpine H2 Boot Script ===" >/dev/console
```

```
# Configure network interface eth0
echo "Configuring eth0 static IP..." >/dev/console
ip addr add 192.168.1.2/24 dev eth0
ip link set eth0 up
ip addr show dev eth0 >/dev/console

# Wait for network interface initialization
sleep 1

# Create directory for web documents
echo "Creating web document directory..." >/dev/console
mkdir -p /var/www

# Create a large file (~10MB) for throughput measurement tests
echo "Creating 10MB test file (large.bin)..." >/dev/console
dd if=/dev/urandom of=/var/www/large.bin bs=1M count=10 >/dev/console 2>&1

# Start BusyBox lightweight HTTP server on port 80
echo "Starting BusyBox HTTP server..." >/dev/console
busybox-extras httpd -p 80 -h /var/www

# Notify CML that the boot process has completed
echo "READY" >/dev/console
exit 0
```

Step 3: Starting the Nodes

1. Click the **Start Lab** button (play icon) at the top of the workspace to boot up both nodes.
2. Wait for the nodes to turn green, indicating they are fully active.

---

4. Lab Execution and Impairment Testing

> *[!IMPORTANT]* ***REMINDER: All screenshots, console outputs, and packet captures (.pcap files) must be taken from the SAME active lab session. Mixing screenshots or trace files across different runs or sessions is not permitted.***

Part A: Measuring Link Delay (Latency)

1. **Verify Baseline Connectivity**: Click on the H1 Client node. In the bottom pane, select the **Console** tab and press **Enter** to get a prompt. Ping the H2 server:

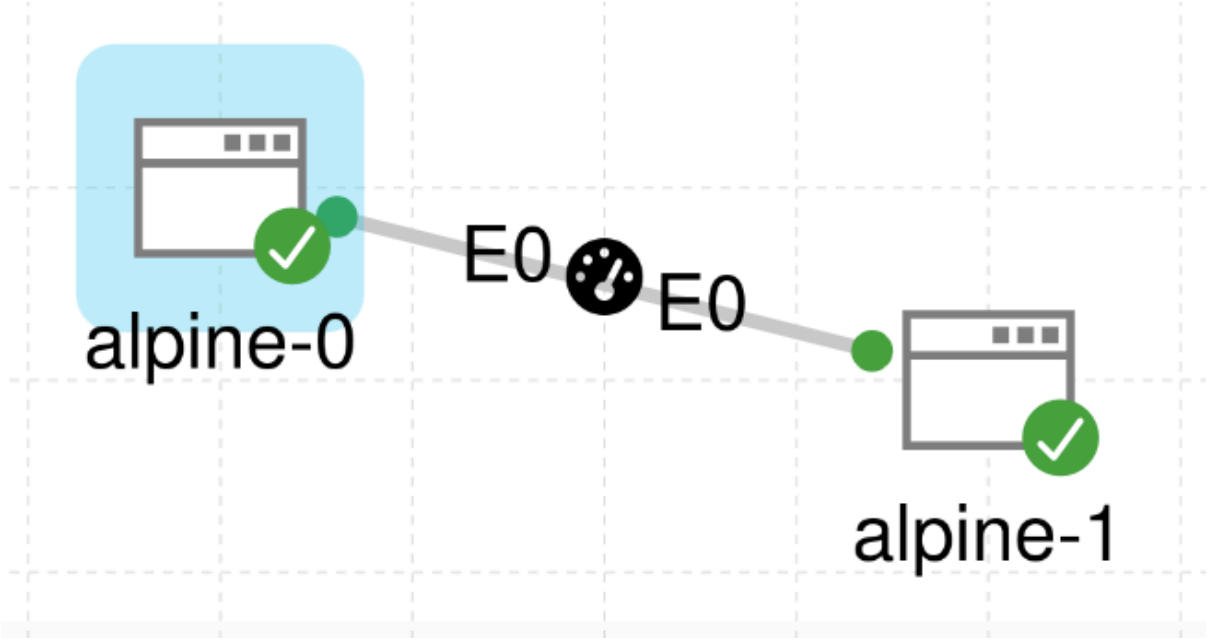
   ```
   ping -c 5 192.168.1.2
   ```

   - **What you should see**: Sub-millisecond Round-Trip Times (typically `< 1 ms`, around `0.1 – 0.5 ms`).
   - *If you don't see this*: If ping fails with `Destination Host Unreachable`, check that both `eth0` interfaces are `UP` and IP addresses (`192.168.1.1` and `192.168.1.2`) are correctly assigned on each node.
   - *Record the minimum, average, and maximum Round-Trip Time (RTT) from the summary.*

2. **Configure 50ms Link Latency**:

   - Click on the **link** connecting H1 and H2 in the CML workspace.

- In the bottom pane, navigate to the **Link Condition** (or **Link Effects**) tab.
- Set the **Latency (Delay)** to `50` ms. Click **Save** or **Update**.
- *Note: CML applies latency on a per-direction basis. A setting of 50ms adds 50ms of delay to packets traveling from H1 -> H2, and 50ms of delay to packets traveling from H2 -> H1.*

> *[!NOTE] **Visual Indicator for Active Link Impairments in CML**: When a link condition (latency, packet loss, or bandwidth limit) is actively applied to a link, CML highlights the link with an impairment symbol/badge directly on the canvas topology, as shown below:*
>
> 
>
>
> *If you do not see this active impairment indicator or your ping/throughput numbers do not change, re-open the Link Condition tab, verify your settings, and click **Update** / **Save**.*

3. **Run 50ms Ping Test**:

```
ping -c 10 192.168.1.2
```

- **What you should see**: The individual ping times and the summary average RTT should now be approximately **~100 ms** (since $50\text{ ms (outbound)} + 50\text{ ms (inbound)} = 100\text{ ms round-trip}$).
- *If you don't see this*: If your RTT is still `< 1 ms` or only `50 ms`, the link condition was not saved or applied to the active link. Click on the link again, verify that `50` ms is entered in the Latency box, click **Save** / **Update**, and re-run the ping test.
- *Record the new minimum, average, and maximum RTT.*

4. **Increase Latency to 150ms**:

- Click on the link again and change the **Latency** to `150` ms. Click **Save** / **Update**.
- Run the ping test once more:

```
ping -c 10 192.168.1.2
```

- **What you should see**: The average RTT should now be approximately **~300 ms** ($150\text{ ms} \times 2 = 300\text{ ms}$).
- *If you don't see this*: Check the link settings in CML, click **Save** / **Update**, and re-run the test.
- *Record the RTT summary.*
- **Reset the Link Condition**: Under the **Link Condition** tab on the link, click the **Reset** button (or set Latency back to `0` ms) and click **Save** / **Update** before proceeding to Part B. Verify that a quick 3-packet ping returns to `< 1 ms`.

Part B: Measuring Packet Loss

1. **Configure Link Packet Loss**:
   - Click on the link. Under the **Link Condition** tab, set the **Packet Loss** to `10` %. Click **Save** / **Update**.
2. **Start Packet Capture**:
   - Click on the link between H1 and H2.
   - Navigate to the **Packet Capture** tab and click **Start**.
3. **Run the 50-Packet Ping Test**:
   - In the H1 Client console, send 50 ping requests:

     ```
     ping -c 50 192.168.1.2
     ```

   - **What you should see**: In the final summary line (`--- 192.168.1.2 ping statistics ---`), you should see approximately **5% to 15% packet loss** (typically ~3 to 8 packets dropped out of 50).
   - *If you don't see this*: If packet loss reports `0% packet loss` , the impairment was not applied to the active link. Re-open the **Link Condition** tab, verify `10` % is set for Packet Loss, click **Save** / **Update**, and re-run the 50-packet ping test.
   - *Record the exact percentage of packet loss reported in the ping summary.*
4. **Generate TCP Traffic under Loss**:
   - Run a download test of the large file:

     ```
     wget -O /dev/null http://192.168.1.2/large.bin
     ```

   - **What you should see**: The download progress bar will exhibit visible stuttering, pauses, or reduced transfer rates compared to a lossless link as TCP detects dropped packets and retransmits segments.
   - *If you don't see this*: If the download finishes instantly with zero pauses, check that the packet loss setting is still active, click **Save** / **Update**, and re-run `wget` .
5. **Download the Capture**:
   - Go to the link's **Packet Capture** tab, click **Stop**, and then click **Download** to save the `.pcap` file.
   - **Reset the Link Condition**: Under the **Link Condition** tab on the link, click the **Reset** button (or set Packet Loss back to `0` %) and click **Save** / **Update** before proceeding to Part C.

Part C: Measuring Throughput (Bandwidth Constraints)

Understanding the `wget` Benchmarking Arguments

To isolate pure network throughput from disk I/O bottlenecks, use the following `wget` command:

```
wget -O /dev/null http://192.168.1.2/large.bin
```

Flag & Argument Breakdown:

- `-O /dev/null` : Routes the downloaded data directly to the Linux null device (`/dev/null`) instead of writing to disk. This ensures your measured speeds reflect network link capacity rather than virtual disk write speeds.
- `http://192.168.1.2/large.bin` : Fetches the 10MB test file hosted by the server's HTTP daemon.

> *[!TIP]* ***Optional - Using `time` for Exact Elapsed Duration****: You can prefix the command with* `time` *to measure the exact elapsed time down to the millisecond:*
>
> ```
> time wget -O /dev/null http://192.168.1.2/large.bin
> ```
>
> *The* `real` *time reported in the output gives you the exact total transit duration in seconds.*

Step-by-Step Execution:

1. **Configure 10 Mbps Link Bandwidth**:
    - Click on the link. Under the **Link Condition** tab, set the **Bandwidth** (or throughput limit) to `10` Mbps (or `10000` kbps depending on CML version). Click **Save** / **Update**.
2. **Start Packet Capture for Throughput**:
    - Click on the link between H1 and H2.
    - Navigate to the **Packet Capture** tab and click **Start**.
3. **Run the 10 Mbps Download Test**:
    - In H1's console, run:

      ```
      wget -O /dev/null http://192.168.1.2/large.bin
      ```

    - **What you should see**: The transfer speed reported by `wget` should stabilize around **~1.1 to 1.25 MB/s** ($10\text{ Mbps} / 8\text{ bits per Byte} = 1.25\text{ MB/s}$).
    - *If you don't see this*: If `wget` reports an excessively high speed (e.g. `50+ MB/s` or `100+ MB/s`), the bandwidth cap was not applied. Click on the link, check the Bandwidth field, ensure `10` Mbps is set, click **Save** / **Update**, and re-run the `wget` download.
    - *Record the average transfer speed (in MB/s or KB/s) and capture a screenshot of the completed wget command.*
4. **Stop and Download Throughput Capture**:
    - Go to the link's **Packet Capture** tab, click **Stop**, and then click **Download** to save the capture file. Name it `throughput_10Mbps_test.pcap` .
5. **Restrict Link Bandwidth to 500 kbps**:
    - Click on the link and set the **Bandwidth** limit to `500` kbps. Click **Save** / **Update**.
    - Run the download test again:

      ```
      wget -O /dev/null http://192.168.1.2/large.bin
      ```

- **What you should see**: The transfer speed should drop significantly to **~55 to 62 KB/s** ($500\text{ kbps} / 8 = 62.5\text{ KB/s}$), and the download progress bar will advance much more slowly.
- *If you don't see this*: If the speed is still reporting in MB/s, verify the bandwidth unit dropdown is set to `kbps` rather than `Mbps`, click **Save** / **Update**, and re-run the download test.
- *Record the new average transfer speed and capture a screenshot of the wget progress.*

6. **Stop and Download Throughput Capture**:
   - Go to the link's **Packet Capture** tab, click **Stop**, and then click **Download** to save the capture file. Name it `throughput_500Kbps_test.pcap`.
7. **Reset Link Impairments**: Under the **Link Condition** tab on the link, click the **Reset** button and click **Save** / **Update** to restore default full bandwidth, 0ms latency, and 0% loss.

---

5. Wireshark Analysis and Questions

Open your downloaded packet captures from **Part B** (`packet_loss.pcap`) and **Part C** (`throughput_test.pcap`) in Wireshark and answer the following questions.

> ***IMPORTANT - Annotation Requirement****: For questions 3, 5, and 8, you must include a screenshot of the relevant packets or graphs in Wireshark and* ***annotate it*** *(e.g., using arrows, circles, or text callouts) to clearly point out the packets, fields, throughput values, flags, or sequence numbers you are describing. Handing in unannotated screenshots will result in a point deduction.*

5.1 Latency and RTT (Part A)

1. What is the relationship between the configured one-way link latency in CML and the actual Round-Trip Time (RTT) observed in your ping results? Why does this relationship exist?
2. In Part A, did the RTT remain perfectly constant, or was there variation (jitter)? Explain what factors could introduce RTT variation in a virtual/emulated environment.

5.2 Packet Loss Analysis (Part B)

3. Filter `packet_loss.pcap` in Wireshark for `icmp`. Look at the sequence numbers of the ICMP Echo Requests (`icmp.seq`). Do you see gaps in the sequence numbers? Find one missing request and provide an **annotated screenshot** pointing out the packet numbers immediately before and after the gap.
4. When an ICMP Echo Request is lost due to packet loss, does the server send an Echo Reply? Explain using your Wireshark observations.
5. In Wireshark, filter your capture for TCP analysis flags using `tcp.analysis.flags` (or use individual filters: `tcp.analysis.retransmission` , `tcp.analysis.duplicate_ack` , and `tcp.analysis.out_of_order`). Locate examples of the following three phenomena and provide **annotated screenshots** clearly labeling each:
   - **TCP Retransmissions** (`[TCP Retransmission]` or `[TCP Fast Retransmission]`): Point out the original frame/sequence number and the retransmitted segment.
   - **TCP Duplicate ACKs** (`[TCP Dup ACK]`): Point out where the receiver repeatedly acknowledges the last in-order byte while waiting for missing data.
   - **TCP Out-of-Order Segments** (`[TCP Out-of-Order]`): Point out where a segment arrives with a sequence number ahead of the expected byte stream due to prior dropped packets.
6. Explain the technical relationship between out-of-order segments, duplicate ACKs, and TCP Fast Retransmit:
   - Why does an out-of-order segment cause the receiving TCP stack to immediately generate a duplicate ACK?
   - How many duplicate ACKs must the sender receive before triggering a Fast Retransmit, and why does this prevent waiting for the Retransmission Timeout (RTO) timer to expire?

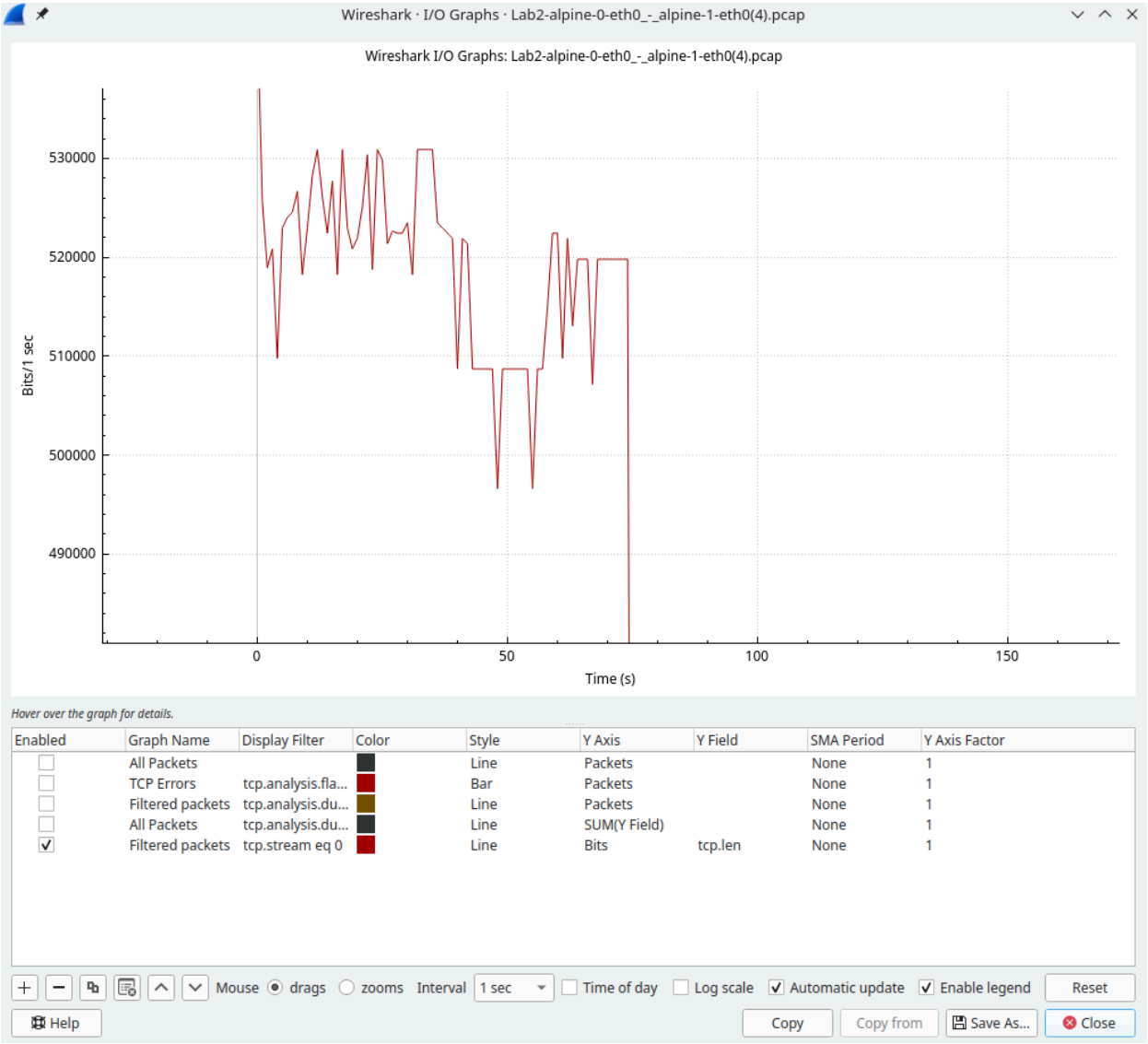
7. How does packet loss affect the overall transmission time of a TCP stream compared to a lossless link? Refer to your `wget` results.

5.3 Throughput & Bandwidth Analysis (Part C)

8. **Option A: Graphing Rate/Throughput (Bits/s or Bytes/s)**
   To generate the graph showing bandwidth limits plateauing:

   1. Open `throughput_test.pcap` in Wireshark.
   2. Select a packet from the download stream, right-click, and select **Follow $\rightarrow$ TCP Stream** (or filter by `tcp.stream == X`).
   3. Navigate to **Statistics $\rightarrow$ I/O Graphs**.
   4. In the bottom-left, click the **+** button to add a graph trace.
   5. Set the parameters:
      - **Y-Field**: `tcp.len` (or leave blank for frame length `frame.len`)
      - **Y-Axis / Function**: Select **BITS/s** or **BYTES/s** (or `SUM(tcp.len) * 8` depending on your Wireshark version).

   The graph will display bandwidth over time, clearly showing flat plateaus where the throughput hits the 10 Mbps or 500 kbps limits. Refer to the example below for what your graph should look like:



Provide an **annotated screenshot** of your own graph showing the TCP download stream and pointing out the plateau regions.

9. Compare your download speeds from Part C. When the bandwidth limit was `10 Mbps`, what was the actual download speed reported by `wget` in MB/s? When the limit was `500 kbps`, what was the speed in KB/s? Convert your download speeds to bits per second ($\text{Speed in bps} = \text{Speed in Bytes/sec} \times 8$) and construct a comparison table showing:

    - Configured Link Bandwidth (bps)
    - Observed `wget` Download Rate (Bytes/s)
    - Calculated Throughput (bps)
    - Efficiency Ratio ($\frac{\text{Calculated Throughput}}{\text{Configured Bandwidth}} \times 100\%$)

10. Explain the difference between **throughput** and **bandwidth**. Which one represents the theoretical maximum capacity of the link, and which one represents the actual observed rate of successful data delivery? Why is measured throughput almost always slightly lower than theoretical link bandwidth (consider packet encapsulation overhead)?

---

6. Troubleshooting

Issue Encountered: [Describe what went wrong, if anything]
Discovery Method: [How did you identify the issue?]
Resolution: [How did you resolve it?]

---

7. What to Hand In

Submit a lab report containing:

1. Topology & Setup Deliverables

- A screenshot of your CML topology showing the connected H1 and H2 hosts.

2. Part A Deliverables (Latency)

- Console output screenshots of the ping RTT summaries for 0ms, 50ms, and 150ms delays on H1.
- Answers to Analysis Questions 1 and 2.

3. Part B Deliverables (Packet Loss)

- Console output screenshot of the ping summary showing the packet loss percentage on H1.
- The downloaded **`packet_loss.pcap`** capture file.
- Annotated screenshots from Wireshark showing:
  - The gap in ICMP sequence numbers (from Question 3).
  - The **TCP Retransmission**, **TCP Duplicate ACK**, and **TCP Out-of-Order** packet flags (from Question 5).
- Answers to Analysis Questions 3, 4, 5, 6, and 7.

4. Part C Deliverables (Throughput & Bandwidth Constraints)

- Console output screenshots of the `wget` download speed results for both **10 Mbps** and **500 kbps** bandwidth limits.
- The downloaded **`throughput_test.pcap`** capture file.
- Annotated Wireshark I/O Graph or TCP Throughput graph screenshot showing the bandwidth plateaus (from Question 8).
- The Bandwidth vs. Throughput conversion and comparison table (from Question 9).
- Answers to Analysis Questions 8, 9, and 10.